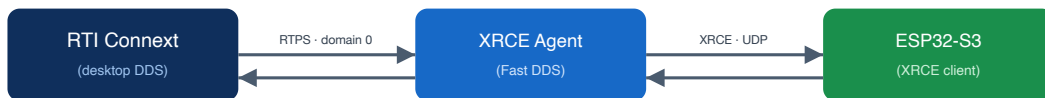


EMBEDDED · DDS · CROSS-VENDOR INTEROP

Smart-Home Node over DDS

ESP32-S3 · eProxima Micro XRCE-DDS · RTI Connex



Technical Project Report

Target: ESP32-S3-DevKitC-1 · ESP-IDF v5.4 · RTI Connex 7.7.0

Generated 9 July 2026

1. Overview & Objective

This project implements a **smart-home actuator node** on an ESP32-S3 that is controlled and monitored over a real **DDS (Data Distribution Service)** dataspace, demonstrating **cross-vendor interoperability** between three independent DDS stacks that all share a single IDL type contract.

The device drives two dimmable LEDs and a positionable servo. Commands and telemetry travel over DDS domain 0. The headline goal — proven end-to-end — is that **RTI Connex** (a commercial desktop DDS) can command and observe the ESP32, even though the device itself runs **eProsima Micro XRCE-DDS** and is bridged by an agent backed by **eProsima Fast DDS**.

Capabilities delivered

- **LED1 / LED2** — independent PWM brightness control (0–255), via LEDC.
- **Servo** — angular position control (0–180°), via the MCPWM peripheral.
- **Command path** — any DDS publisher → agent → ESP32 → hardware.
- **Telemetry path** — ESP32 publishes `device_state` (live LED/servo values) back to the domain.
- **Cross-vendor** — RTI Connex and eProsima participants interoperate on the same domain from one shared `smart_home.idl`.
- **Web dashboard** — a local browser control panel (sliders & buttons) with a live device-state readout, so no command line is needed for everyday control.

Result: All objectives met and verified on hardware. RTI Connex publishes `led_cmd` / `servo_cmd` and subscribes `device_state`; the device applies commands and reports state, with the telemetry read back matching the commands exactly.

2. System Architecture

Three processes participate in one logical DDS domain. The ESP32 speaks the lightweight **XRCE** protocol to an **Agent**; the agent owns real DDS entities on the device's behalf and bridges them onto the RTPS wire, where full DDS participants (RTI Connex, or the eProsima host tools) discover and exchange data with them.

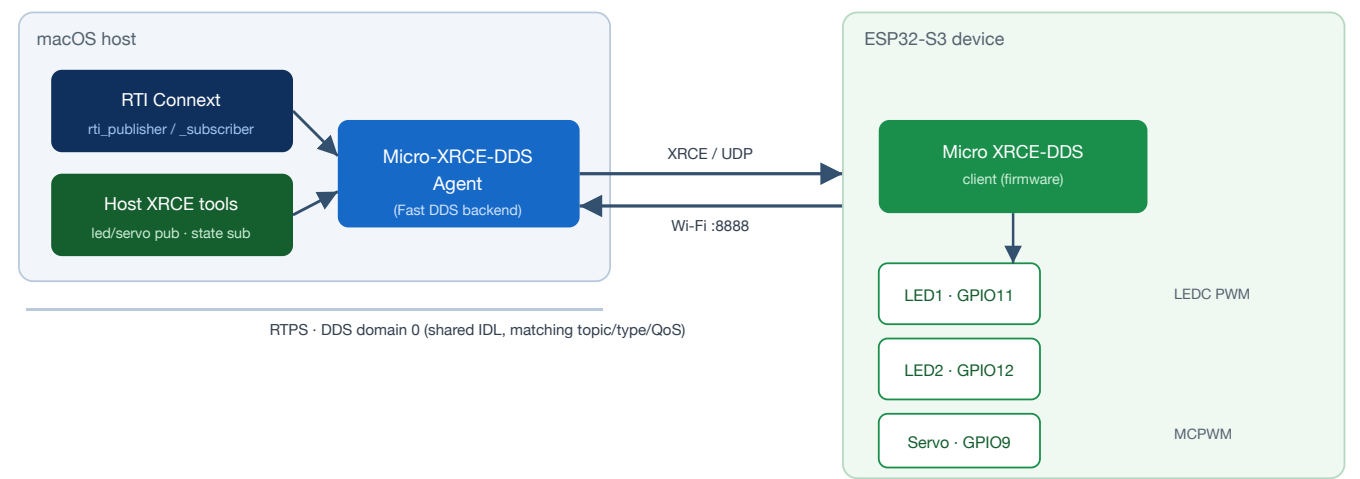


Figure 1 — End-to-end architecture. The agent runs natively on the host so its DDS is on the same network as RTI Connex.

Why the agent runs natively (not in Docker): On macOS, Docker Desktop's UDP forwarding is unreliable — the ESP32 (on a phone-hotspot LAN) often can't reach a Docker agent's `:8888`, and it never bridges the dynamic RTPS ports a host-native RTI participant needs. Running the agent as a native macOS process (it binds the Mac's real `:8888`) fixes both: the board reaches it directly, and RTI can discover its DDS. The project therefore uses the **native agent for everything** (dashboard and RTI); the Docker agent is only a fallback.

3. Technology Stack

Layer	Technology	Role in this project
MCU	ESP32-S3 (DevKitC-1)	Runs the firmware; drives LEDs (LEDC) and servo (MCPWM); Wi-Fi station.
Firmware SDK	ESP-IDF v5.4	Native build system (<code>idf.py</code> + CMake), FreeRTOS, drivers, lwIP.
Device DDS	eProsima Micro XRCE-DDS Client v3.0.1	Lightweight DDS-XRCE client vendored as an IDF component; UDP transport.
Serialization	Micro CDR v2.0.2	CDR encode/decode of the IDL types on the device.
Bridge	Micro XRCE-DDS Agent v3.0.1	Creates real DDS entities for the client; backed by Fast DDS; RTPS on domain 0.
Desktop DDS	RTI Connxt 7.7.0	Cross-vendor participant: commands and monitors the device.
Type contract	OMG IDL (<code>smart_home.idl</code>)	Single source of truth; both sides generate CDR-compatible type support.
Code generators	microxrccddsgen / rtiddsgen	Generate device-side and RTI-side type support from the same IDL.
Host tools	Native C (Micro XRCE-DDS) & C++11 (RTI)	Publishers/subscribers on the host for control & telemetry.

4. The Interop Contract (IDL)

Every participant is generated from this one file. Matching *domain*, *topic name*, *type name*, *CDR layout*, and QoS is what makes cross-vendor communication possible.

```
module smart_home {
    // Command TO the device: id 1->LED1, 2->LED2 ; brightness 0..255
    struct LedCommand { unsigned short id; unsigned short brightness; };
    // Command TO the device: servo angle 0..180
    struct ServoCommand { unsigned short angle; };
    // Telemetry FROM the device: last applied values
    struct DeviceState { unsigned short led1; unsigned short led2; unsigned short servo_angle; };
};
// topic "led_cmd" : smart_home::LedCommand (device subscribes)
// topic "servo_cmd" : smart_home::ServoCommand (device subscribes)
// topic "device_state" : smart_home::DeviceState (device publishes)
```

Cross-vendor interop checklist (all satisfied & verified)

Requirement	Value	Status
Domain ID	0	✓
Topic names	led_cmd · servo_cmd · device_state	✓
Type names	smart_home::LedCommand / ServoCommand / DeviceState	✓
CDR layout	identical (generated from same IDL)	✓
QoS	RELIABLE (device streams & RTI writer/reader)	✓
Discovery	native agent on host network; verified with <code>rtiddsspy</code>	✓

5. Repository Structure

```
esp32_dds_control/
├── smart_home_dds/
│   ├── CMakeLists.txt / sdkconfig.defaults
│   ├── smart_home.idl
│   ├── main/
│   │   ├── smart_home_client.c
│   │   ├── wifi_sta.c/.h
│   │   ├── smart_home.c/.h
│   │   ├── idf_component.yml
│   │   ├── CMakeLists.txt
│   │   ├── components/eprosima/
│   │   ├── host_tools/
│   │   ├── led_publisher.c · servo_publisher.c · state_subscriber.c
│   │   └── build.sh
│   └── components/eprosima/
│       ├── smart_home.idl
│       ├── smart_home.{hpp,cxx} + smart_homePlugin.*
│       ├── rti_publisher.cxx · rti_subscriber.cxx
│       └── build.sh
├── dashboard/
│   ├── run.sh
│   ├── server.py
│   └── index.html
├── docs/
└── .gitignore
```

Build outputs (`build/` , `managed_components/` , compiled binaries) are git-ignored and regenerated by the build scripts.

6. Hardware & Circuit

Signal	ESP32-S3 pin	Peripheral	Wiring
LED1	GPIO 11	LEDC ch0 (5 kHz, 13-bit PWM)	GPIO → 220 Ω → LED anode; LED cathode → GND
LED2	GPIO 12	LEDC ch1 (5 kHz, 13-bit PWM)	GPIO → 220 Ω → LED anode; LED cathode → GND
Servo signal	GPIO 9	MCPWM (50 Hz, 544–2400 μs)	→ servo orange wire
Servo power	—	external 5 V	servo red → external 5 V supply
Common ground	GND	—	servo brown + LED cathodes + ext-5 V GND all tied to ESP32 GND

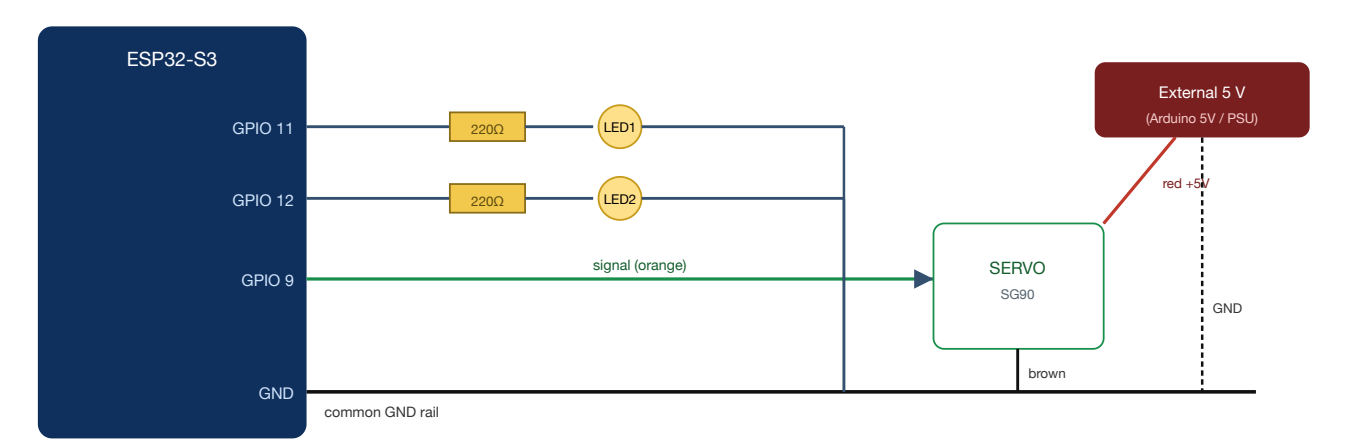


Figure 2 — Wiring. LEDs are driven directly from GPIO through a 220 Ω resistor to a common ground. The servo takes its signal from GPIO 9 but its **power from an external 5 V source**, with all grounds tied together.

Power note: An SG90 servo needs ~5 V and meaningful current; it will not move reliably on 3.3 V (or a 3.7 V cell). It is powered from an external 5 V supply while its signal ground shares the ESP32 ground — this shared ground is mandatory for the PWM signal to be interpreted.

7. Firmware Design

The firmware (`smart_home_client.c`) initialises PWM, connects Wi-Fi, creates its XRCE session and DDS entities via the agent, then runs a loop that applies inbound commands and publishes telemetry once per second.

7.1 Actuator drivers

LEDs use the LEDC peripheral (8-bit brightness mapped to 13-bit duty). The servo uses the **MCPWM** peripheral, generating a standard 50 Hz / 544–2400 μ s pulse (the Arduino `Servo` calibration, so labelled angles match the physical position) — chosen over LEDC to fully isolate servo timing from the LED timers. It boots to 90° (safe centre); the two pulse-width constants are tunable per servo.

```
static void apply_led(ledc_channel_t ch, uint16_t brightness) {
    if (brightness > 255) brightness = 255;
    uint32_t duty = (uint32_t)brightness * 8191u / 255u;    // 8-bit -> 13-bit
    ledc_set_duty(LMODE, ch, duty);
    ledc_update_duty(LMODE, ch);
}

static void apply_servo(uint16_t angle) {                  // MCPWM comparator = pulse width in  $\mu$ s
    if (angle > 180) angle = 180;
    uint32_t us = SERVO_MIN_US + ((uint32_t)(SERVO_MAX_US - SERVO_MIN_US) * angle) / 180;
    if (servo_cmp) mcpwm_comparator_set_compare_value(servo_cmp, us);    // 544..2400  $\mu$ s
}
```

7.2 Receiving commands (XRCE topic callback)

```
static void on_topic(uxrSession *s, uxrObjectId oid, ... ucdrBuffer *ub, ...) {
    if (oid.id == LED_DR_ID) {
        smart_home_LedCommand c; smart_home_LedCommand_deserialize_topic(ub, &c);
        uint16_t b = c.brightness > 255 ? 255 : c.brightness;
        if (c.id == 1) { apply_led(LED1_CH, b); g_led1 = b; }
        else if (c.id == 2) { apply_led(LED2_CH, b); g_led2 = b; }
    } else if (oid.id == SERVO_DR_ID) {
        smart_home_ServoCommand c; smart_home_ServoCommand_deserialize_topic(ub, &c);
        uint16_t a = c.angle > 180 ? 180 : c.angle;
        apply_servo(a); g_servo = a;
    }
}
```

7.3 Publishing telemetry (once per second)

```
smart_home_DeviceState st = { g_led1, g_led2, g_servo };
ucdrBuffer mb;
uint32_t sz = smart_home_DeviceState_size_of_topic(&st, 0);
if (uxr_prepare_output_stream(&session, out, st_dw, &mb, sz))
    smart_home_DeviceState_serialize_topic(&mb, &st);    // flushed by uxr_run_session
```

Two real firmware bugs found & fixed: (1) the servo LEDC timer originally used a 16-bit resolution — invalid on the ESP32-S3, whose LEDC maxes at 14-bit (later moved to MCPWM entirely); (2) the XRCE output stream buffer was only 512 B, silently truncating the entity-creation burst so the servo's `DataReader` was never created — enlarged to `MTU × 8` so all entities register.

8. Host Tools & RTI Application

eProsima host tools (C)

A second XRCE client on the host to command/monitor without RTI.

```
// led_publisher: create writer on led_cmd,
// then publish a LedCommand
smart_home_LedCommand cmd;
cmd.id = 1; cmd.brightness = 200;
uxr_prepare_output_stream(&s, out, dw,
    &ub, size);
smart_home_LedCommand_serialize_topic(
    &ub, &cmd);
```

RTI Connex participant (C++11)

Native DDS participant on domain 0 — no device changes needed.

```
dds::domain::DomainParticipant part(0);
dds::topic::Topic<smart_home::LedCommand>
    topic(part, "led_cmd");
dds::pub::DataWriter<...> w(pub, topic, qos);
smart_home::LedCommand cmd;
cmd.id = 1; cmd.brightness = 200;
w.write(cmd);
```

Both are generated from the same `smart_home.idl` (via `microxrcceddsgen` and `rtiddsgen` respectively), guaranteeing identical CDR layout and hence interoperability.

9. Build & Run Guide

Step 0 — Prerequisites (one-time)

- **ESP-IDF v5.4** installed; activated per shell via `source ~/esp/idf_env.sh` (uses a Python 3.12 environment).
- **Native Micro XRCE-DDS Agent** — used for everything (dashboard *and* RTI). Built once from Micro-XRCE-DDS-Agent source with `-DUAGENT_SOCKETCAN_PROFILE=OFF` on macOS (the Linux CAN transport won't compile). Docker is only an optional fallback; its UDP forwarding to the LAN is unreliable on macOS.
- **Host XRCE tools** link against the Micro-XRCE-DDS-Client + Micro-CDR built for the host (installed at `~/esp/eprosima-host`, built once from `smart_home_dds/components/eprosima/`); `host_tools/build.sh` expects it there.
- **RTI Connex 7.7.0** with a valid license (only for the RTI path).

Step 1 — Configure & flash the firmware

```
source ~/esp/idf_env.sh          # activate ESP-IDF (Python 3.12 env)
cd smart_home_dds
# set WIFI_SSID / WIFI_PASS and AGENT_IP (= the Mac's IP) in main/smart_home_client.c
idf.py set-target esp32s3        # first time only
idf.py build
ls /dev/cu.usbmodem*            # find the port, then:
idf.py -p /dev/cu.usbmodemXXXX flash
# then press the board's RST button to launch the app
```

Step 2 — Start the Agent (native)

One agent, used for both the dashboard and RTI. The dashboard launcher (`run.sh`, Step 3) starts it automatically, so you usually don't run this by hand — but to start it manually:

```
# native agent — binds the Mac's real :8888 (reliable; Docker UDP forwarding is not on macOS)
DYLD_LIBRARY_PATH=$(ls -d ~/esp/Micro-XRCE-DDS-Agent/build/temp_install/*/lib | tr '\n' ':') \
~/esp/Micro-XRCE-DDS-Agent/build/MicroXRCEAgent udp4 -p 8888 -v6
```

Only ONE agent may bind UDP 8888 at a time. (Docker fallback, if ever needed: `docker run -d --name xrce_agent --platform linux/amd64 -p 8888:8888/udp microros/micro-ros-agent:jazzy udp4 --port 8888 -v6` — but on this setup the board often can't reach it.)

Step 3 (recommended) — Web dashboard

The simplest way to drive the device: a browser control panel — no terminal commands. One launcher builds the tools, starts the **native agent** if needed, and opens the page:

```
bash dashboard/run.sh          # then press the board's RST button once
```

Open <http://localhost:8080>. Move a slider or click a button → the command travels over DDS to the device → the *live panel* shows the state the device reports back (its `device_state` telemetry). A status indicator turns green when the device is online. An **Activity Log** panel records the full round-trip with timestamps — commands sent (→) and device state received (←) — so both directions are visible without touching the command line.



Figure 3 — The web control panel. Sliders/buttons issue DDS commands; the lower panel mirrors the device's live telemetry.

Under the hood, the page talks to a tiny local Python bridge (`dashboard/server.py` , standard library only) that **reuses the host tools** — button/slider actions run `led_publisher` / `servo_publisher` , and a background `state_subscriber` feeds the live panel. No extra installs, and it rides on the same agent + firmware.

Step 3 (advanced) — Command line

eProsimia host tools (build once with `host_tools/build.sh`):

```
cd smart_home_dds/host_tools
./led_publisher 127.0.0.1 8888 1 255
./led_publisher 127.0.0.1 8888 2 40
./servo_publisher 127.0.0.1 8888 90
./state_subscriber 127.0.0.1 8888
```

RTI Connex (build once with `rti_app/build.sh` ; needs the native agent running):

```
export NDDSHOME=/Applications/rti_connex_dds-7.7.0
export PATH=$NDDSHOME/bin:$PATH # for rtiddsspy/rtiddsgen
export DYLD_LIBRARY_PATH=$NDDSHOME/lib/arm64Darwin23clang16.0
cd rti_app
./rti_publisher led 1 200 # apps use UDPv4 only (shmem disabled)
./rti_publisher servo 90
./rti_subscriber
rtiddsspy -domainId 0 -transport 1 # -transport 1 = UDPv4 only (skip shmem)
```

Session-volatile settings (re-check if the device stops connecting): Wi-Fi SSID/password, and `AGENT_IP` — the Mac's IP can change on a phone hotspot (`ipconfig getifaddr en0`). Update, rebuild, reflash, and press RST.

10. Verification Results

The full loop was confirmed on hardware. With RTI publishing commands and RTI subscribing to `device_state` , the reported state tracked the commands exactly:

```
RTI publishes LED1=222, LED2=90, servo=30 ; then LED1=0, servo=150

device_state (as RTI's subscriber received it):
LED1=0    LED2=0    servo=90    <- boot default
LED1=222  LED2=0    servo=90
LED1=222  LED2=90   servo=90
LED1=222  LED2=90   servo=30
LED1=0    LED2=90   servo=30
LED1=0    LED2=90   servo=150
```

`rtiddsspy -domainId 0` independently confirmed discovery of the device's endpoints with exact type-name matches: a `writer` on `device_state` and `readers` on `led_cmd` / `servo_cmd` , all typed `smart_home::*` .

11. Engineering Notes & Lessons Learned

Area	Note
------	------

ESP32-S3 LEDC	Max PWM resolution is 14-bit (not 16-bit). Servo later moved to MCPWM for clean 50 Hz timing.
XRCE buffers	Stream buffers must hold the full entity-creation burst; undersized buffers silently drop later entities.
Agent image	The <code>eprosima/micro-xrce-dds-agent</code> image does not exist; use <code>microros/micro-ros-agent</code> (<code>--platform linux/amd64</code> on Apple Silicon).
Agent transport	Docker Desktop's UDP forwarding to the LAN is unreliable on macOS — the ESP32 gets "agent unreachable" despite correct IP/Wi-Fi. Run the agent natively (binds the Mac's real :8888) for both the dashboard and RTI. Docker also never bridges the RTPS ports RTI needs.
RTI on macOS	RTI's default shared-memory transport fails on macOS (small shmem limits) — disable it and use UDPv4 only: set the participant transport mask in code (done in <code>rti_publisher/subscriber</code>), and run <code>rtiddspy</code> with <code>-transport 1</code> .
Servo power	SG90 needs ~5 V; 3.3–3.7 V will not move it. Use an external 5 V supply with a shared ground.
Servo calibration	Pulse range 544–2400 μ s (Arduino <code>Servo</code> mapping); wider ranges peg the SG90 stops and skew the angle scale. SG90s are $\pm 10^\circ$ imprecise — tune the two constants if needed.
Dashboard	Local Python bridge reuses the host tools; commands sent with <code>repeat=2</code> for reliable delivery. An Activity Log panel shows both directions (sent $\rightarrow$ / received $\leftarrow$). If telemetry shows a commanded value but an LED is dark, it's wiring, not code.
Board reset	Opening the serial port can reset the DevKitC-1 into download mode; the physical RST button reliably launches the app.
Wiring	Most "not working" symptoms were breadboard contact issues; direct pin-to-component wiring is reliable.

Appendix A — Environment Setup (from a bare machine)

The run guide (§9) assumes these one-time installs. The *repository already contains* the vendored device client, the hand-authored `config.h` files, and the generated type support — so a fresh clone builds the firmware as-is. What must be installed on the machine are the toolchains and the host/desktop DDS pieces below. Paths match this project's layout (`~/esp/...`).

A.1 — ESP-IDF v5.4 (with a Python 3.12 shim)

ESP-IDF v5.4 does not support Python 3.14 (this machine's default). A small shim points `python3` at 3.12 for IDF only.

```
mkdir -p ~/esp && cd ~/esp
git clone -b v5.4 --recursive https://github.com/espressif/esp-idf.git

# shim: make python3 resolve to 3.12 for the IDF install/activate
mkdir -p ~/esp/pyshim
ln -sf "$(pyenv root)/versions/3.12.9/bin/python3.12" ~/esp/pyshim/python3
ln -sf ~/esp/pyshim/python3 ~/esp/pyshim/python
cd ~/esp/esp-idf && PATH="$HOME/esp/pyshim:$PATH" ./install.sh esp32s3

# activation helper — create ~/esp/idf_env.sh with:
#   export PATH="$HOME/esp/pyshim:$PATH"; . "$HOME/esp/esp-idf/export.sh"
# then in any shell: source ~/esp/idf_env.sh
```

A.2 — Type generator (microxrccddsgen)

Only needed to (re)generate the device-side type support from the IDL. Requires Java + Gradle.

```
cd ~/esp
git clone --recurse-submodules https://github.com/eProsima/Micro-XRCE-DDS-Gen.git
cd Micro-XRCE-DDS-Gen && ./gradlew assemble
# regenerate (only if smart_home.idl changes):
cd smart_home_dds && ~/esp/Micro-XRCE-DDS-Gen/scripts/microxrccddsgen -replace -d main smart_home.idl
```

A.3 — Host eProsima libraries (for the host tools & dashboard)

The host publishers/subscriber and the dashboard link against the Micro-XRCE-DDS-Client + Micro-CDR built *for the host*, installed at `~/esp/eProsima-host` (superbuild off).

```
SRC=smart_home_dds/components/eProsima # vendored sources (in the repo)
cmake -S $SRC/Micro-CDR -B /tmp/ucdr -DUCDR_SUPERBUILD=OFF \
-DMAKE_INSTALL_PREFIX=~/.esp/eProsima-host && cmake --build /tmp/ucdr --target install
cmake -S $SRC/Micro-XRCE-DDS-Client -B /tmp/ucl -DUCLIENT_SUPERBUILD=OFF -DUCLIENT_BUILD_MICROCDR=OFF \
-DMAKE_PREFIX_PATH=~/.esp/eProsima-host -DMAKE_INSTALL_PREFIX=~/.esp/eProsima-host \
&& cmake --build /tmp/ucl --target install
# (merge microcdr's include/ + lib/ into ~/.esp/eProsima-host so build.sh finds them)
bash smart_home_dds/host_tools/build.sh # builds led/servo publisher + state subscriber
```

A.4 — Native XRCE Agent (required for RTI interop)

Built from source so its DDS runs on the host network. The Linux CAN transport must be disabled on macOS.

```
brew install asio tinyxml2 openssl@3
cd ~/esp
git clone -b v3.0.1 https://github.com/eProsima/Micro-XRCE-DDS-Agent.git
cd Micro-XRCE-DDS-Agent && mkdir build && cd build
cmake .. -DUAGENT_BUILD_EXECUTABLE=ON -DUAGENT_SOCKETCAN_PROFILE=OFF \
-DMAKE_BUILD_TYPE=Release -DOPENSSL_ROOT_DIR="$(brew --prefix openssl@3)"
cmake --build . -j4 # produces build/MicroXRCEAgent
```

A.5 — RTI Connex 7.7.0 (for the RTI participant)

- Install via RTI Launcher (free trial / university licence); place `rti_license.dat`. Target `arm64Darwin23clang16.0` (native Apple Silicon).
- Generate + build the RTI app from the same IDL:

```
export NDDSHOME=/Applications/rti_connex_dds-7.7.0
export PATH=$NDDSHOME/bin:$PATH
cd rti_app
```

```
rtiddsgen -language C++11 -platform arm64Darwin23clang16.0 smart_home.idl # types (in repo)
bash build.sh # builds rti_publisher + rti_subscriber
```

A.6 — Docker (optional fallback only)

Not required — the project uses the native agent (A.4). Docker Desktop is only an optional fallback agent, and on this macOS setup its UDP forwarding to the LAN is unreliable, so the board often can't reach it. Image: `microros/micro-ros-agent:jazzy` (`--platform linux/amd64` on Apple Silicon).

Summary of one-time installs: ESP-IDF v5.4 (+3.12 shim) → firmware; microxrceiddsgen → device types; `~/esp/eprosima-host` → host tools; **native agent** → **both dashboard & RTI**; RTI Connex 7.7.0 → RTI participant. After these, everyday use is just `bash dashboard/run.sh` + `RST`.

— End of report —